一個問題，一條線

預測的價值＝完美預測比最好的不預測做法好多少；每一章回答上一章留下的問題

1 問題與賭注

問　預測能讓使用者回來時更快嗎？

答　**近年一批研究押注預測；我們一開始也這樣想**

▼ 預測的好處，最多有多大？

2 最直接的版本：寫入時預知

問　寫入時就依位置放好，有用嗎？

答　**沒用：16 組比較 0 組贏（TTFT 差 −0.37～+1.16%）**

位置回來時一樣看得到；最前面的塊回來時多半靠重算，不從儲存讀

▼ 預測只剩「回來時」。怎麼公平地量它？

3 比較對象決定答案

問　預測值多少，取決於跟誰比？

答　**是。同一個完美預知：只能重算 15.9% → 有慢層 3.4% → 再加 Cake 2.6%**

FreeInference 前 6 週、CPU 32 GiB、慢層 6.98 GiB/s、GPU 閒、對 LRU（模擬；另掃容量與頻寬）

▼ 所以先把「不預測」做到最強

4 把「不預測」的做法做到最強

問　最好的不預測做法是什麼？

答　**在 Cake 上加 INT4：最慢那層 12.1 秒 → 1.24 秒；CPU 層看解壓寫法（差 6.8 倍）**

Cake 是別人已發表的方法，不是我們的；為了公平，不是主角

▼ 不預測的做法做好了，預測還剩多少？

5 在最強的不預測做法之上量上限

問　完美預測最多再省多少？

答　**FreeInference ≤0.30%；Mooncake 預取 50.5% → 14.6%（128→1,024 GiB）**

取決於固定規則已命中多少（FreeInference 99.29%；Mooncake 沒算）

▼ Mooncake 的大上限是真的嗎？

6 大的上限還剩多少

問　簡單的打字線索能拿到多少？

答　**打字提示拿到完美預取改善的 0.81；完美預取只比它快 4.0%**

不是主流做法，是「不用預測」的參考點；含排隊上限再掉一半以上

▼ 沒有這種簡單線索的部分呢？

7 結論

問　預測值不值得？

答　**兩份紀錄上有限：FreeInference 幾乎沒有；對話上打字線索拿到大部分**

待解：淘汰順序、Mooncake toolagent、Mooncake 命中率、真的預測器

實測：寫入時放置、INT4（在 Cake 上）、解壓、打字提示（GPU 縮小重播）。模擬：15.9%→2.6%、完美預測上限、含排隊。